SENATI

Escuela de Tecnologías de la Información

SOLUCIONARIO

Solucionario Guía 14-15

Árboles y Recursividad

Solución paso a paso de los ejercicios propuestos y de las preguntas de reflexión

Escuela	Tecnologías de la Información
Carrera	Ingeniería de Software con Inteligencia Artificial
Módulo formativo	Artificial Intelligence with Machine Learning in Java (Oracle Academy)
Semestre	IV
Semana	14 - 15
Documento	Solucionario de los ejercicios propuestos de la Guía 14-15
Herramientas	Java 21 (JDK) · Visual Studio Code · Extension Pack for Java
Instructor	Rembrandt Ubalde Enríquez
Uso	Material del instructor

Contenido

Indicaciones generales 3
Ejercicio 1. Contar e invertir un texto con recursividad 4
Paso 1. Pensar en forma recursiva 4
Paso 2. Programa 4
Paso 3. Traza de invertir("abc") 5
Ejercicio 2. Torres de Hanói 6
Paso 1. Idea recursiva 6
Paso 2. Programa 6
Paso 3. Explicación 7
Ejercicio 3. Nuevos métodos para el árbol binario 8
Paso 1. Idea de cada método 8
Paso 2. Clase ArbolBinario con los métodos nuevos 8
Paso 3. Programa de prueba 10
Paso 4. Verificación con la Figura 1 de la guía 10
Ejercicio 4. Árbol de la expresión (5 − 1) * (2 + 3) + 4 11
Paso 1. Dibujar el árbol 11
Paso 2. Programa 11
Paso 3. Comprobación a mano 12
Ejercicio 5. Juego Sí/No mejorado 13
Paso 1. Crear el árbol de carreras 13
Paso 2. Programa 13
Paso 3. Partida de ejemplo 15
Paso 4. Explicación 16
Respuestas a las preguntas de reflexión 18

(Si el índice aparece vacío, haz clic derecho sobre él y elige «Actualizar campos».)

Indicaciones generales

Cada ejercicio se resuelve en un archivo nuevo dentro de la carpeta `src` del proyecto **Semana14_15_Arboles_Java**. Los nombres de los archivos empiezan con `Ej` seguido del número del ejercicio, para no confundirlos con las prácticas.

Ejercicio	Archivo(s)	Necesita
1	`Ej01_CadenasRecursivas.java`	—
2	`Ej02_Hanoi.java`	—
3	`ArbolBinario.java` (métodos nuevos) y `Ej03_ArbolExtendido.java`	`Nodo.java`
4	`Ej04_NuevaExpresion.java`	`Nodo.java` y `P05_ArbolExpresion.java`
5	`Ej05_JuegoMejorado.java` y `datos/carreras.txt`	`Nodo.java` y `datos/animales.txt`

Todos los resultados de este solucionario se obtuvieron **ejecutando el código con Java 21**; no son aproximados.

Ejercicio 1. Contar e invertir un texto con recursividad

Enunciado: Escribe dos métodos recursivos: `contar(String texto, char c)`, que cuente cuántas veces aparece un carácter, e `invertir(String texto)`, que devuelva el texto al revés. Prueba con "inteligencia artificial".

Paso 1. Pensar en forma recursiva

Método	Caso base	Caso recursivo
`contar`	Texto vacío → 0	(¿el primer carácter es c? 1 : 0) + contar(resto del texto)
`invertir`	Texto de 0 o 1 letra → el mismo texto	invertir(resto del texto) + primer carácter

En ambos casos el «resto del texto» es `texto.substring(1)`, que tiene una letra menos. Así cada llamada se acerca al caso base.

Paso 2. Programa

Archivo: src/Ej01_CadenasRecursivas.java

```
// Ej01_CadenasRecursivas.java - Contar un carácter e invertir un texto con recursividad
public class Ej01_CadenasRecursivas {

  // contar = (¿el primer carácter es c? 1 : 0) + contar en el resto del texto
  static int contar(String texto, char c) {
    if (texto.isEmpty()) return 0;                          // caso base: texto vacío
    int primero = (texto.charAt(0) == c) ? 1 : 0;
    return primero + contar(texto.substring(1), c);         // caso recursivo
  }

  // invertir = invertir(resto del texto) + primer carácter
  static String invertir(String texto) {
    if (texto.length() <= 1) return texto;                  // caso base
    return invertir(texto.substring(1)) + texto.charAt(0);  // caso recursivo
  }

  public static void main(String[] args) {
    String frase = "inteligencia artificial";
    for (char c : new char[]{'i', 'a', 'l', 'z'}) {
      System.out.println("La letra '" + c + "' aparece " + contar(frase, c) + " veces");
    }
    System.out.println("Texto original : " + frase);
    System.out.println("Texto invertido: " + invertir(frase));
  }
}
```

▶ Resultado esperado en la terminal

```
La letra 'i' aparece 6 veces
La letra 'a' aparece 3 veces
La letra 'l' aparece 2 veces
La letra 'z' aparece 0 veces
Texto original : inteligencia artificial
Texto invertido: laicifitra aicnegiletni
```

Paso 3. Traza de invertir("abc")

Cómo se resuelve

```
invertir("abc") = invertir("bc") + 'a'
                = (invertir("c") + 'b') + 'a'
                = ("c" + 'b') + 'a'          <- caso base: "c"
                = "cba"
```

Ejercicio 2. Torres de Hanói

Enunciado: Resuelve las Torres de Hanói de forma recursiva para 3 discos, mostrando cada movimiento. Verifica que el total de movimientos sea $2^n - 1$.

Paso 1. Idea recursiva

Para mover **n** discos de A a C usando B como auxiliar:

1. Mover los **n – 1** discos de arriba de A a B (usando C como auxiliar): es el **mismo problema**, pero más pequeño.
2. Mover el disco más grande (el n) de A a C.
3. Mover los **n – 1** discos de B a C (usando A como auxiliar).

El **caso base** es n = 0: no hay nada que mover.

Paso 2. Programa

Archivo: src/Ej02_Hanoi.java

```
// Ej02_Hanoi.java - Torres de Hanói resueltas con recursividad
public class Ej02_Hanoi {

  static int movimientos = 0;

  // Mueve n discos de "origen" a "destino" usando "auxiliar"
  static void hanoi(int n, char origen, char destino, char auxiliar) {
    if (n == 0) return;                                  // caso base: nada que mover
    hanoi(n - 1, origen, auxiliar, destino);             // 1) n-1 discos al auxiliar
    movimientos++;
    System.out.printf("%d) Mover disco %d de %c a %c%n",
        movimientos, n, origen, destino);
    hanoi(n - 1, auxiliar, destino, origen);             // 2) n-1 discos al destino
  }

  public static void main(String[] args) {
    int n = 3;
    hanoi(n, 'A', 'C', 'B');
    System.out.println("Total de movimientos: " + movimientos
        + " | 2^" + n + " - 1 = " + ((int) Math.pow(2, n) - 1));

    System.out.println("\nDiscos  Movimientos (2^n - 1)");
    for (int d = 1; d <= 6; d++) {
      System.out.printf("  %d      %d%n", d, (int) Math.pow(2, d) - 1);
    }
  }
}
```

▶ Resultado esperado en la terminal

```
1) Mover disco 1 de A a C
2) Mover disco 2 de A a B
3) Mover disco 1 de C a B
4) Mover disco 3 de A a C
5) Mover disco 1 de B a A
6) Mover disco 2 de B a C
7) Mover disco 1 de A a C
Total de movimientos: 7 | 2^3 - 1 = 7

Discos  Movimientos (2^n - 1)
  1      1
  2      3
  3      7
  4      15
  5      31
  6      63
```

Paso 3. Explicación

- Con 3 discos se necesitan **7 movimientos**, que coincide con $2^3 - 1 = 7$.
- Cada disco adicional **duplica** el trabajo más uno: $M(n) = 2 \cdot M(n - 1) + 1$. Con 64 discos (la leyenda original) se necesitarían más de 18 trillones de movimientos.
- El problema parece difícil de programar con bucles, pero con recursividad se resuelve en 4 líneas.

Ejercicio 3. Nuevos métodos para el árbol binario

Enunciado: Agrega a `ArbolBinario` los métodos recursivos `suma()`, `minimo()`, `maximo()` y `nodosEnNivel(int k)`. Pruébalos con los valores de la Práctica 03.

Paso 1. Idea de cada método

Método	Caso base	Caso recursivo
`suma`	Nodo nulo → 0	dato + suma(izquierdo) + suma(derecho)
`minimo`	No tiene hijo izquierdo → su dato	minimo(izquierdo): en un ABB el menor está a la izquierda
`maximo`	No tiene hijo derecho → su dato	maximo(derecho)
`nodosEnNivel`	Nodo nulo → nada; k = 1 → agrega el dato	Baja a ambos hijos con k − 1

Paso 2. Clase ArbolBinario con los métodos nuevos

Se muestra la clase con `insertar`, `inOrden` y los cuatro métodos del ejercicio, que puedes agregar al final de tu `ArbolBinario.java` de la Práctica 03.

Archivo: src/ArbolBinario.java

```
// ArbolBinario.java - Versión de la Práctica 03 con los métodos del Ejercicio 3
import java.util.ArrayList;
import java.util.List;

public class ArbolBinario {
  private Nodo<Integer> raiz;

  public void insertar(int valor) { raiz = insertar(raiz, valor); }
  private Nodo<Integer> insertar(Nodo<Integer> n, int valor) {
    if (n == null) return new Nodo<>(valor);
    if (valor < n.dato) n.izquierdo = insertar(n.izquierdo, valor);
    else if (valor > n.dato) n.derecho = insertar(n.derecho, valor);
    return n;
  }

  public List<Integer> inOrden() {
    List<Integer> lista = new ArrayList<>();
    inOrden(raiz, lista);
    return lista;
  }
  private void inOrden(Nodo<Integer> n, List<Integer> l) {
    if (n == null) return;
    inOrden(n.izquierdo, l);
    l.add(n.dato);
    inOrden(n.derecho, l);
  }

  // ==================== NUEVOS MÉTODOS (Ejercicio 3) ====================

  // SUMA: mi valor + la suma de mi izquierda + la suma de mi derecha
  public int suma() { return suma(raiz); }
  private int suma(Nodo<Integer> n) {
    if (n == null) return 0;
    return n.dato + suma(n.izquierdo) + suma(n.derecho);
  }

  // MÍNIMO: en un ABB está en el nodo de más a la IZQUIERDA
  public int minimo() { return minimo(raiz); }
  private int minimo(Nodo<Integer> n) {
    if (n.izquierdo == null) return n.dato;      // caso base: no hay nada más a la izq.
    return minimo(n.izquierdo);
  }

  // MÁXIMO: en un ABB está en el nodo de más a la DERECHA
  public int maximo() { return maximo(raiz); }
  private int maximo(Nodo<Integer> n) {
    if (n.derecho == null) return n.dato;
    return maximo(n.derecho);
  }

  // NODOS EN EL NIVEL k (la raíz está en el nivel 1)
  public List<Integer> nodosEnNivel(int k) {
    List<Integer> lista = new ArrayList<>();
    nodosEnNivel(raiz, k, lista);
    return lista;
  }
  private void nodosEnNivel(Nodo<Integer> n, int k, List<Integer> l) {
    if (n == null) return;
    if (k == 1) {                                // caso base: llegamos al nivel pedido
      l.add(n.dato);
      return;
    }
    nodosEnNivel(n.izquierdo, k - 1, l);         // bajar un nivel = restar 1 a k
    nodosEnNivel(n.derecho, k - 1, l);
  }
}
```

Paso 3. Programa de prueba

Archivo: src/Ej03_ArbolExtendido.java

```
// Ej03_ArbolExtendido.java - Prueba de suma, mínimo, máximo y nodos por nivel
public class Ej03_ArbolExtendido {
  public static void main(String[] args) {
    ArbolBinario arbol = new ArbolBinario();
    for (int v : new int[]{50, 30, 70, 20, 40, 60, 80, 35}) arbol.insertar(v);

    System.out.println("Valores (in-order): " + arbol.inOrden());
    System.out.println("Suma    : " + arbol.suma());
    System.out.println("Mínimo  : " + arbol.minimo());
    System.out.println("Máximo  : " + arbol.maximo());
    for (int k = 1; k <= 5; k++) {
      System.out.println("Nivel " + k + " : " + arbol.nodosEnNivel(k));
    }
  }
}
```

▶ Resultado esperado en la terminal

```
Valores (in-order): [20, 30, 35, 40, 50, 60, 70, 80]
Suma    : 385
Mínimo  : 20
Máximo  : 80
Nivel 1 : [50]
Nivel 2 : [30, 70]
Nivel 3 : [20, 40, 60, 80]
Nivel 4 : [35]
Nivel 5 : []
```

Paso 4. Verificación con la Figura 1 de la guía

- Suma: 50 + 30 + 70 + 20 + 40 + 60 + 80 + 35 = **385**.
- El mínimo (20) es el nodo de más a la izquierda y el máximo (80), el de más a la derecha.
- Los niveles coinciden con la figura. En el nivel 5 la lista está **vacía** porque la altura del árbol es 4.
- `minimo` y `maximo` solo recorren **un camino**, no todo el árbol: por eso son muy rápidos en un ABB.

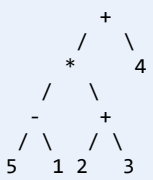
Ejercicio 4. Árbol de la expresión (5 − 1) * (2 + 3) + 4

Enunciado: Construye el árbol de la expresión `(5 - 1) * (2 + 3) + 4`. Muestra su notación prefija y postfija, y evalúalo con el método de la Práctica 05.

Paso 1. Dibujar el árbol

El operador que se calcula **al final** va en la raíz. Aquí es el `+`, porque primero se calcula el producto y al final se suma 4. Debajo va el `*`, cuyos hijos son las dos restas y sumas de los paréntesis:

Árbol de la expresión



Paso 2. Programa

No hace falta volver a escribir los métodos: `prefija`, `infija`, `postfija` y `evaluar` de la Práctica 05 son `static` y se pueden usar desde otra clase del mismo proyecto.

Archivo: src/Ej04_NuevaExpresion.java

```
// Ej04_NuevaExpresion.java - Árbol de la expresión (5 - 1) * (2 + 3) + 4
public class Ej04_NuevaExpresion {
  public static void main(String[] args) {
    //                +
    //              /   \
    //             *     4
    //           /   \
    //          -     +
    //         / \   / \
    //        5   1 2   3
    Nodo<String> expr = new Nodo<>("+",
        new Nodo<>("*",
            new Nodo<>("-", new Nodo<>("5"), new Nodo<>("1")),
            new Nodo<>("+", new Nodo<>("2"), new Nodo<>("3"))),
        new Nodo<>("4"));

    // Reutilizamos los métodos estáticos de la Práctica 05
    System.out.println("Prefija  (pre-order) : " + P05_ArbolExpresion.prefija(expr));
    System.out.println("Infija   (in-order)  : " + P05_ArbolExpresion.infija(expr));
    System.out.println("Postfija (post-order): " + P05_ArbolExpresion.postfija(expr));
    System.out.println("\nEvaluación (post-order):");
    System.out.println("Resultado = " + P05_ArbolExpresion.evaluar(expr));
  }
}
```

▶ **Resultado esperado en la terminal**

```
Prefija  (pre-order) : + * - 5 1 + 2 3 4
Infija   (in-order)  : (((5 - 1) * (2 + 3)) + 4)
Postfija (post-order): 5 1 - 2 3 + * 4 +

Evaluación (post-order):
   calcula 5.0 - 1.0 = 4.0
   calcula 2.0 + 3.0 = 5.0
   calcula 4.0 * 5.0 = 20.0
   calcula 20.0 + 4.0 = 24.0
Resultado = 24.0
```

Paso 3. Comprobación a mano

(5 − 1) = 4; (2 + 3) = 5; 4 × 5 = 20; 20 + 4 = **24**. La evaluación sigue el orden **post-order**: primero las hojas y al final la raíz.

Ejercicio 5. Juego Sí/No mejorado

Enunciado: Modifica el Juego Sí/No para que: a) muestre la lista de todos los animales que conoce, b) muestre la altura del árbol y c) cambie de tema a «Adivina la carrera de SENATI» con un archivo `carreras.txt`.

Paso 1. Crear el árbol de carreras

Archivo: datos/carreras.txt

```
P:¿Te gusta programar?
P:¿Te interesa crear sistemas que aprenden de los datos?
R:Ingeniería de Software con Inteligencia Artificial
R:Desarrollo de Software
P:¿Prefieres trabajar con máquinas y equipos?
R:Mecatrónica Industrial
R:Diseño Gráfico Digital
```

> 💡 **Consejo:** Los estudiantes pueden usar otras carreras y preguntas. Lo importante es respetar el formato: cada `P:` debe tener exactamente dos nodos debajo, primero el «sí» y luego el «no».

Paso 2. Programa

Requisito	Solución	Técnica
a) Listar respuestas	Método `listar`: si el nodo es hoja lo agrega a la lista; si no, baja a sus dos hijos.	Recorrido pre-order de las hojas
b) Altura	Método `altura`: 1 + la mayor altura de sus hijos.	Recursión
c) Cambiar de tema	Se pregunta el tema al inicio y se elige el archivo; la variable `tema` adapta los mensajes.	Un menú con `switch`

Archivo: src/Ej05_JuegoMejorado.java

```
// Ej05_JuegoMejorado.java - Juego Sí/No con lista de respuestas, altura y temas
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.Scanner;

public class Ej05_JuegoMejorado {

  static Scanner sc = new Scanner(System.in);
  static String tema;          // "animal" o "carrera": se usa en los mensajes

  static Nodo<String> cargar(Iterator<String> lineas) {
    String linea = lineas.next();
    Nodo<String> n = new Nodo<>(linea.substring(2));
    if (linea.startsWith("P:")) {
      n.izquierdo = cargar(lineas);
      n.derecho = cargar(lineas);
    }
    return n;
  }

  static void guardar(Nodo<String> n, PrintWriter out) {
    if (n.esHoja()) {
      out.println("R:" + n.dato);
    } else {
      out.println("P:" + n.dato);
      guardar(n.izquierdo, out);
      guardar(n.derecho, out);
    }
  }

  // a) LISTA de respuestas: recorre el árbol y guarda solo las HOJAS
  static void listar(Nodo<String> n, List<String> lista) {
    if (n.esHoja()) {
      lista.add(n.dato);
      return;
    }
    listar(n.izquierdo, lista);
    listar(n.derecho, lista);
  }

  // b) ALTURA: 1 + la mayor altura de sus dos subárboles
  static int altura(Nodo<String> n) {
    if (n == null) return 0;
    return 1 + Math.max(altura(n.izquierdo), altura(n.derecho));
  }

  static void jugar(Nodo<String> n) {
    if (!n.esHoja()) {
      if (preguntar(n.dato)) jugar(n.izquierdo);
      else jugar(n.derecho);
      return;
    }
    if (preguntar("¿Tu " + tema + " es " + n.dato + "?")) {
      System.out.println("¡Adiviné!");
    } else {
      System.out.print("Me rindo. ¿Cuál era? ");
      String nuevo = sc.nextLine().trim();
      System.out.print("Escribe una pregunta que diferencie " + nuevo + " de "
          + n.dato + ": ");
      String pregunta = sc.nextLine().trim();
      boolean si = preguntar("Para " + nuevo + ", ¿la respuesta es sí?");
      Nodo<String> viejo = new Nodo<>(n.dato), otro = new Nodo<>(nuevo);
      n.dato = pregunta;
```

```
      n.izquierdo = si ? otro : viejo;
      n.derecho = si ? viejo : otro;
      System.out.println("¡Gracias! Aprendí: " + nuevo);
    }
  }

  static boolean preguntar(String texto) {
    System.out.print(texto + " (s/n): ");
    return sc.nextLine().trim().toLowerCase().startsWith("s");
  }

  static int menu() {
    System.out.println("\n1) Jugar  2) Ver respuestas  3) Ver altura  4) Salir");
    System.out.print("Opción: ");
    return Integer.parseInt(sc.nextLine().trim());
  }

  public static void main(String[] args) throws IOException {
    // c) El tema se elige al inicio: cada tema usa su propio archivo
    System.out.print("Tema (1 = animales, 2 = carreras de SENATI): ");
    boolean carreras = sc.nextLine().trim().equals("2");
    tema = carreras ? "carrera" : "animal";
    String archivo = carreras ? "datos/carreras.txt" : "datos/animales.txt";
    Nodo<String> raiz = cargar(Files.readAllLines(Path.of(archivo)).iterator());

    int op;
    while ((op = menu()) != 4) {
      switch (op) {
        case 1 -> jugar(raiz);
        case 2 -> {
          List<String> lista = new ArrayList<>();
          listar(raiz, lista);
          System.out.println("Conozco " + lista.size() + ":");
          for (String r : lista) System.out.println("  - " + r);
        }
        case 3 -> System.out.println("Altura del árbol: " + altura(raiz)
            + " niveles (máximo " + (altura(raiz) - 1) + " preguntas)");
        default -> System.out.println("Opción no válida");
      }
    }
    try (PrintWriter out = new PrintWriter(Files.newBufferedWriter(Path.of(archivo)))) {
      guardar(raiz, out);
    }
    System.out.println("Árbol guardado en " + archivo);
  }
}
```

Paso 3. Partida de ejemplo

Se eligió el tema de carreras, se consultó la lista y la altura, y luego se le enseñó una carrera nueva (**Electricidad Industrial**):

▶ Partida de ejemplo

```
Tema (1 = animales, 2 = carreras de SENATI): 2

1) Jugar  2) Ver respuestas  3) Ver altura  4) Salir
Opción: 2
Conozco 4:
  - Ingeniería de Software con Inteligencia Artificial
  - Desarrollo de Software
  - Mecatrónica Industrial
  - Diseño Gráfico Digital

1) Jugar  2) Ver respuestas  3) Ver altura  4) Salir
Opción: 3
Altura del árbol: 3 niveles (máximo 2 preguntas)

1) Jugar  2) Ver respuestas  3) Ver altura  4) Salir
Opción: 1
¿Te gusta programar? (s/n): n
¿Prefieres trabajar con máquinas y equipos? (s/n): s
¿Tu carrera es Mecatrónica Industrial? (s/n): n
Me rindo. ¿Cuál era? Electricidad Industrial
Escribe una pregunta que diferencie Electricidad Industrial de Mecatrónica Industrial:
¿Te interesan los circuitos eléctricos?
Para Electricidad Industrial, ¿la respuesta es sí? (s/n): s
¡Gracias! Aprendí: Electricidad Industrial

1) Jugar  2) Ver respuestas  3) Ver altura  4) Salir
Opción: 2
Conozco 5:
  - Ingeniería de Software con Inteligencia Artificial
  - Desarrollo de Software
  - Electricidad Industrial
  - Mecatrónica Industrial
  - Diseño Gráfico Digital

1) Jugar  2) Ver respuestas  3) Ver altura  4) Salir
Opción: 3
Altura del árbol: 4 niveles (máximo 3 preguntas)

1) Jugar  2) Ver respuestas  3) Ver altura  4) Salir
Opción: 4
Árbol guardado en datos/carreras.txt
```

Archivo `datos/carreras.txt` después de jugar:

datos/carreras.txt

```
P:¿Te gusta programar?
P:¿Te interesa crear sistemas que aprenden de los datos?
R:Ingeniería de Software con Inteligencia Artificial
R:Desarrollo de Software
P:¿Prefieres trabajar con máquinas y equipos?
P:¿Te interesan los circuitos eléctricos?
R:Electricidad Industrial
R:Mecatrónica Industrial
R:Diseño Gráfico Digital
```

Paso 4. Explicación

- Antes de jugar el árbol tiene **4 respuestas** y altura 3: se adivina con a lo más 2 preguntas. Después de aprender, tiene **5 respuestas** y altura 4.
- La altura indica el **número máximo de preguntas + 1**. Un árbol bien equilibrado con n respuestas necesita alrededor de $\log_2(n)$ preguntas.

- El mismo código sirve para cualquier tema: solo cambia el archivo de datos. Es la idea de separar el **algoritmo** del **conocimiento**, como en los sistemas expertos.

Respuestas a las preguntas de reflexión

1. ¿Qué pasa si el caso base es n == 0 y se llama con un número negativo?

Con `factorial(-1)` la siguiente llamada es `factorial(-2)`, luego `factorial(-3)`… el valor se **aleja** de 0 y nunca llega al caso base. El programa se llama sin fin hasta llenar la pila y termina con **StackOverflowError**. Por eso el caso base de la guía es `n <= 1`, que también detiene los valores negativos. Lo ideal es validar el dato antes de llamar a la función.

2. ¿Por qué los árboles son estructuras naturalmente recursivas?

Porque **cada hijo de un nodo es, a su vez, la raíz de un árbol más pequeño** (un subárbol). Un problema sobre el árbol completo se resuelve con el mismo problema sobre sus subárboles: contar los nodos del árbol es 1 + contar los del subárbol izquierdo + contar los del derecho. El caso base es el árbol vacío (`null`).

3. ¿Por qué se guarda el juego en pre-order y no en in-order?

En pre-order cada **pregunta se escribe antes que sus respuestas**. Al leer el archivo, el programa ve primero la raíz, sabe si es pregunta (`P:`) o respuesta (`R:`) y, si es pregunta, lee recursivamente sus dos hijos: el árbol se reconstruye exactamente igual. En in-order la raíz queda en medio de la lista y no se sabe cuál es, así que no se puede reconstruir la estructura.

4. ¿En qué se parece el juego a un sistema de diagnóstico?

Un sistema que diagnostica fallas de una computadora también hace preguntas de sí o no («¿Enciende la luz de encendido?», «¿Se escucha el ventilador?») y cada respuesta descarta posibilidades hasta llegar a un diagnóstico (una hoja: «Falla la fuente de poder»). Ambos son **sistemas expertos** basados en un árbol de decisión; el juego, además, **aprende** nuevos casos cuando se equivoca.